## 2 What happens when you call nlp?

What does spaCy do when you call `nlp` on a string of text?

```
doc = nlp("This is a sentence.")
```

- ◯ Run the tagger, parser and entity recognizer and then the tokenizer.
- ◉ Tokenize the text and apply each pipeline component in order.
- ◯ Connect to the spaCy server to compute the result and return it.
- ◯ Initialize the language, add the pipeline and load in the binary model weights.

Submit

**That's correct!** The tokenizer turns a string of text into a `Doc` object. spaCy then applies every component in the pipeline on document, in order.

✓ Completed Next

## 3 Inspecting the pipeline

Let's inspect the small English pipeline!

- Load the `en_core_web_sm` pipeline and create the `nlp` object.
- Print the names of the pipeline components using `nlp.pipe_names`.
- Print the full pipeline of `(name, component)` tuples using `nlp.pipeline`.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")

print(nlp.pipe_names)
print(nlp.pipeline)
```

Run code Submit

OUTPUT

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec object at 0x780cc311be20>), ('tagge
r', <spacy.pipeline.tagger.Tagger object at 0x780cc311bbe0>), ('parser', <spacy.
pipeline.dep_parser.DependencyParser object at 0x780cc97bd1c0>), ('attribute_rul
er', <spacy.pipeline.attributeruler.AttributeRuler object at 0x780cc2ea44c0>),
('lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer object at 0x780cc2ebc
e00>), ('ner', <spacy.pipeline.ner.EntityRecognizer object at 0x780cc97bd0e0>)]
✔ Well done! Whenever you're unsure about the current pipeline, you can
inspect it by printing nlp.pipe_names or nlp.pipeline.
```

Show hints Show solution Reset

✓ Completed Next

## 5 Use cases for custom components

Which of these problems can be solved by custom pipeline components? Choose all that apply!

1. Updating the trained pipelines and improving their predictions
2. Computing your own values based on tokens and their attributes
3. Adding named entities, for example based on a dictionary
4. Implementing support for an additional language

- ○ 1 and 2.
- ○ 1 and 3.
- ○ 1 and 4.
- ◉ 2 and 3.
- ○ 2 and 4.
- ○ 3 and 4.

Submit

**That's correct!** Custom components are great for adding custom values to documents, tokens and spans, and customizing the `doc.ents`.

✓ Completed Next

## 6 Simple components

The example shows a custom component that prints the number of tokens in a document. Can you complete it?

- Complete the component function with the `doc`'s length.
- Add the `"length_component"` to the existing pipeline as the **first** component.
- Try out the new pipeline and process any text with the `nlp` object – for example "This is a sentence.".

INPUT

```
import spacy
from spacy.language import Language

@Language.component("length_component")
def length_component_function(doc):
    print(f"This document is {len(doc)} tokens long.")
    return doc

nlp = spacy.load("en_core_web_sm")
nlp.add_pipe("length_component", first=True)

doc = nlp("This is a sentence.")
```

Run code | Submit

OUTPUT

```
This document is 5 tokens long.
✔ Perfect! Now let's take a look at a slightly more complex
component!
```

Show hints | Show solution | Reset

✓ Completed | Next

## 7 Complex components

In this exercise, you'll be writing a custom component that uses the `PhraseMatcher` to find animal names in the document and adds the matched spans to the `doc.ents`. A `PhraseMatcher` with the animal patterns has already been created as the variable `matcher`.

- Define the custom component and apply the `matcher` to the `doc`.
- Create a `Span` for each match, assign the label ID for `"ANIMAL"` and overwrite the `doc.ents` with the new spans.
- Add the new component to the pipeline *after* the `"ner"` component.
- Process the text and print the entity text and entity label for the entities in `doc.ents`.

INPUT

```
import spacy
from spacy.language import Language
from spacy.matcher import PhraseMatcher
from spacy.tokens import Span

nlp = spacy.load("en_core_web_sm")

animals = ["Golden Retriever", "cat", "turtle", "Rattus norvegicus"]
matcher = PhraseMatcher(nlp.vocab)
matcher.add("ANIMAL", list(nlp.pipe(animals)))

@Language.component("animal_component")
def animal_component_function(doc):
    matches = matcher(doc)

    spans = [
        Span(doc, start, end, label="ANIMAL")
        for match_id, start, end in matches
    ]

    doc.ents = spans
    return doc

nlp.add_pipe("animal_component", after="ner")

doc = nlp("I have a cat and a Golden Retriever")

print([(ent.text, ent.label_) for ent in doc.ents])
```

Run code Submit

OUTPUT

```
[('cat', 'ANIMAL'), ('Golden Retriever', 'ANIMAL')]
✔ Good job! You've built your first pipeline component for rule-based
entity matching.
```

Show hints Show solution Reset

## 9 Setting extension attributes (1)

Let's practice setting some extension attributes.

### Step 1

- Use `Token.set_extension` to register `"is_country"` (default `False`).
- Update it for `"Spain"` and print it for all tokens.

INPUT

```
import spacy
from spacy.tokens import Token

nlp = spacy.blank("en")

Token.set_extension("is_country", default=False)

doc = nlp("I live in Spain.")
doc[3]._.is_country = True

print([(t.text, t._.is_country) for t in doc])
```

Run code  Submit

OUTPUT

```
[('I', False), ('live', False), ('in', False), ('Spain', True), ('.', False)]
✘ Are you printing the right token attributes?
```

Show hints  Show solution  Reset

???

### Step 2

- Use `Token.set_extension` to register `"reversed"` (getter function `get_reversed`).
- Print its value for each token.

INPUT

```
import spacy
from spacy.tokens import Token

nlp = spacy.blank("en")

def get_reversed(token):
    return token.text[::-1]

Token.set_extension("reversed", getter=get_reversed)

doc = nlp("All generalizations are false, including this one.")

for token in doc:
    print(token._.reversed)
```

Run code  Submit

OUTPUT

```
llA
snoitazilareneg
era
eslaf
,
gnidulcni
siht
eno
.
✔ Good job! Let's set some more complex attributes.
```

Show hints  Show solution  Reset

✓ Completed  Next

## 10 Setting extension attributes (2)

Let's try setting some more complex attributes using getters and method extensions.

### Part 1

- Complete the `get_has_number` function.
- Use `Doc.set_extension` to register `"has_number"` (getter `get_has_number`) and print its value.

INPUT

```
import spacy
from spacy.tokens import Doc

nlp = spacy.blank("en")

def get_has_number(doc):
    return any(token.like_num for token in doc)

Doc.set_extension("has_number", getter=get_has_number)

doc = nlp("The museum closed for five years in 2012.")
print(doc._.has_number)
```

Run code | Submit

OUTPUT

```
True
✔ Nice work!
```

Show hints | Show solution | Reset

Part 2

- Use `Span.set_extension` to register `"to_html"` (method `to_html`).
- Call it on `doc[0:2]` with the tag `"strong"`.

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.blank("en")

def to_html(span, tag):
    return f"<{tag}>{span.text}</{tag}>"

Span.set_extension("to_html", method=to_html)

doc = nlp("Hello world, this is a sentence.")
span = doc[0:2]

print(span._.to_html("strong"))
```

Run code | Submit

OUTPUT

```
<strong>Hello world</strong>
✔ Perfect! In the next exercise, you'll get to combine custom
attributes with custom pipeline components.
```

Show hints | Show solution | Reset

✓ Completed | Next

## 11 Entities and extensions

In this exercise, you'll combine custom extension attributes with the statistical predictions and create an attribute getter that returns a Wikipedia search URL if the span is a person, organization, or location.

- Complete the `get_wikipedia_url` getter so it only returns the URL if the span's label is in the list of labels.
- Set the `Span` extension `"wikipedia_url"` using the getter `get_wikipedia_url`.
- Iterate over the entities in the `doc` and output their Wikipedia URL.

INPUT

```
import spacy
from spacy.tokens import Span

nlp = spacy.load("en_core_web_sm")

def get_wikipedia_url(span):
    if span.label_ in ("PERSON", "ORG", "GPE", "LOCATION"):
        name = span.text.replace(" ", "_")
        return "https://en.wikipedia.org/w/index.php?search=" + name

Span.set_extension("wikipedia_url", getter=get_wikipedia_url)

doc = nlp("David Bowie was a famous musician.")

for ent in doc.ents:
    print(ent.text, ent._.wikipedia_url)
```

Run code | Submit

OUTPUT

```
David Bowie https://en.wikipedia.org/w/index.php?search=David_Bowie
✔ Nice! You now have a pipeline component that uses named entities
predicted by the model to generate Wikipedia URLs  and adds them as a custom
attribute. Try opening the link in your browser to see what happens!
```

Show hints | Show solution | Reset

✓ Completed | Next

## 12 Components with extensions

Extension attributes are especially powerful if they're combined with custom pipeline components. In this exercise, you'll write a pipeline component that finds country names and a custom extension attribute that returns a country's capital, if available.

A phrase matcher with all countries is available as the variable `matcher`. A dictionary of countries mapped to their capital cities is available as the variable `CAPITALS`.

- Complete the `countries_component_function` and create a `Span` with the label `"GPE"` (geopolitical entity) for all matches.
- Add the component to the pipeline.
- Register the Span extension attribute `"capital"` with the getter `get_capital`.
- Process the text and print the entity text, entity label and entity capital for each entity span in `doc.ents`.

INPUT

```
import json
import spacy
from spacy.language import Language
from spacy.tokens import Span
from spacy.matcher import PhraseMatcher

with open("exercises/en/countries.json", encoding="utf8") as f:
    COUNTRIES = json.load(f)

with open("exercises/en/capitals.json", encoding="utf8") as f:
    CAPITALS = json.load(f)

nlp = spacy.blank("en")
matcher = PhraseMatcher(nlp.vocab)
matcher.add("COUNTRY", list(nlp.pipe(COUNTRIES)))

@Language.component("countries_component")
def countries_component_function(doc):
    matches = matcher(doc)
    doc.ents = [
        Span(doc, start, end, label="GPE")
        for match_id, start, end in matches
    ]
    return doc

nlp.add_pipe("countries_component")

def get_capital(span):
    return CAPITALS.get(span.text)

Span.set_extension("capital", getter=get_capital)

doc = nlp("Czech Republic may help Slovakia protect its airspace")

for ent in doc.ents:
    print(ent.text, ent.label_, ent._.capital)
```

Run code | Submit

OUTPUT

```
Czech Republic GPE Prague
Slovakia GPE Bratislava
✔ Well done! This is a great example of how you can add structured data
to your spaCy pipeline.
```

Show hints | Show solution | Reset

## 14 Processing streams

In this exercise, you'll be using `nlp.pipe` for more efficient text processing. The `nlp` object has already been created for you. A list of tweets about a popular American fast food chain are available as the variable `TEXTS`.

### Part 1

- Rewrite the example to use `nlp.pipe`. Instead of iterating over the texts and processing them, iterate over the `doc` objects yielded by `nlp.pipe`.

INPUT

```
import json
import spacy

nlp = spacy.load("en_core_web_sm")

with open("exercises/en/tweets.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

# Process the texts and print the adjectives
for doc in nlp.pipe(TEXTS):
    print([token.text for token in doc if token.pos_ == "ADJ"])
```

Run code | Submit

OUTPUT

```
['favorite']
['sick']
[]
['happy']
['delicious', 'fast']
['SANDWICH']
['terrible', 'gettin']
✔ Nice!
```

Show hints | Show solution | Reset

Part 2

- Rewrite the example to use `nlp.pipe`. Don't forget to call `list()` around the result to turn it into a list.

INPUT

```
import json
import spacy

nlp = spacy.load("en_core_web_sm")

with open("exercises/en/tweets.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

# Process the texts and print the entities
docs = list(nlp.pipe(TEXTS))
entities = [doc.ents for doc in docs]
print(*entities)
```

Run code Submit

OUTPUT

```
(McDonalds,) () (McDonalds,) (McDonalds, Spain) (The Arch Deluxe :P,) () (This m
orning,)
✔ Great work!
```

Show solution Reset

Part 3

- Rewrite the example to use `nlp.pipe`. Don't forget to call `list()` around the result to turn it into a list.

INPUT

```
import spacy

nlp = spacy.blank("en")

people = ["David Bowie", "Angela Merkel", "Lady Gaga"]

patterns = list(nlp.pipe(people))
```

Run code  Submit

OUTPUT

```
✔ Good job! Let's move on to a practical example that uses nlp.pipe to
process documents with additional meta data.
```

Show solution  Reset

✓ Completed  Next

## 15 Processing data with context

In this exercise, you'll be using custom attributes to add author and book meta information to quotes.

A list of `[text, context]` examples is available as the variable `DATA`. The texts are quotes from famous books, and the contexts dictionaries with the keys `"author"` and `"book"`.

- Use the `set_extension` method to register the custom attributes `"author"` and `"book"` on the `Doc`, which default to `None`.
- Process the `[text, context]` pairs in `DATA` using `nlp.pipe` with `as_tuples=True`.
- Overwrite the `doc._.book` and `doc._.author` with the respective info passed in as the context.

INPUT

```
import json
import spacy
from spacy.tokens import Doc

with open("exercises/en/bookquotes.json", encoding="utf8") as f:
    DATA = json.load(f)

nlp = spacy.blank("en")

Doc.set_extension("author", default=None)
Doc.set_extension("book", default=None)

for doc, context in nlp.pipe(DATA, as_tuples=True):
    doc._.book = context["book"]
    doc._.author = context["author"]

    print(doc.text)
    print(doc._.book, doc._.author)
```

Run code  Submit

OUTPUT

```
One morning, when Gregor Samsa woke from troubled dreams, he found himself trans
formed in his bed into a horrible vermin.
Metamorphosis Franz Kafka
I know not all that may be coming, but be it what it will, I'll go to it laughin
g.
Moby-Dick or, The Whale Herman Melville
It was the best of times, it was the worst of times.
A Tale of Two Cities Charles Dickens
The only people for me are the mad ones, the ones who are mad to live, mad to ta
lk, mad to be saved, desirous of everything at the same time, the ones who never
yawn or say a commonplace thing, but burn, burn, burn like fabulous yellow roman
candles exploding like spiders across the stars.
On the Road Jack Kerouac
It was a bright cold day in April, and the clocks were striking thirteen.
1984 George Orwell
Nowadays people know the price of everything and the value of nothing.
The Picture Of Dorian Gray Oscar Wilde
✔ Well done! The same technique is useful for a variety of tasks. For
example, you could pass in page or paragraph numbers to relate the processed Doc
back to the position in a larger document. Or you could pass in other structured
data like IDs referring to a knowledge base.
```

## 16 Selective processing

In this exercise, you'll use the `nlp.make_doc` and `nlp.select_pipes` methods to only run selected components when processing a text.

### Part 1

- Rewrite the code to only tokenize the text using `nlp.make_doc`.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Chick-fil-A is an American fast food restaurant chain."

doc = nlp.make_doc(text)

print([token.text for token in doc])
```

Run code | Submit

OUTPUT

```
['Chick', '-', 'fil', '-', 'A', 'is', 'an', 'American', 'fast', 'food', 'restaur
ant', 'chain', '.']
✔ Nicely done!
```

Show hints | Show solution | Reset

### Part 2

- Disable the tagger and lemmatizer using the `nlp.select_pipes` method.
- Process the text and print all entities in the `doc`.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Chick-fil-A is an American fast food restaurant chain."

with nlp.select_pipes(disable=["tagger", "lemmatizer"]):
    doc = nlp(text)
    print(doc.ents)
```

Run code | Submit

OUTPUT

```
(American,)
✔ Perfect! Now that you've practiced the performance tips and tricks,
you're ready for the next chapter and training spaCy's neural network
models.
```

Show hints | Show solution | Reset

## Chapter 3: Processing Pipelines

This chapter will show you everything you need to know about spaCy's processing pipeline. You'll learn what goes on under the hood when you process a text, how to write your own components and add them to the pipeline, and how to use custom attributes to add your own metadata to the documents, spans and tokens.


1. Processing pipelines 
2. What happens when you call nlp?
3. Inspecting the pipeline
4. Custom pipeline components 
5. Use cases for custom components
6. Simple components
7. Complex components
8. Extension attributes 
9. Setting extension attributes (1)
10. Setting extension attributes (2)
11. Entities and extensions
12. Components with extensions
13. Scaling and performance 
14. Processing streams
15. Processing data with context
16. Selective processing